

Using Flan T5 and SQuAD for automatic question answer generation with transfer learning

Amardeep Singh
Enrollment: E23CSEU2189

Rudra Sharma
Enrollment: E23CSEU2163

Nishant Sharma
Enrollment: E23CSEU2170

Abstract—Out here, past typical language jobs, machines that build Q&A pairs spark new ways to shape learning aids. Not simply reading words, they pull out questions - this leap happens through heavy pattern study. Take one adapted form of Google's Flan-T5 core: fine-tuned only on SQuAD examples. Forget responding - it flips logic. Feed it a paragraph and a key phrase, then wait as it shapes fitting prompts around what fits. Pulling grammar apart comes early, thanks to helpers such as NLTK, setting up what follows. Only once parsing finishes does prompting begin, a quiet handoff between steps. Responses grow step by step, shaped by how each piece connects. Hidden underneath are scripts made special, knobs turned just right, decisions drawn from how sequences behave. No tricks live here - just order, information, and small plans doing steady work without noise.

Some work with understanding how people talk. Machines that form questions come into play here too. Built around transformer architecture are certain setups - T5 is one example. There's also a collection of data known as SQuAD doing rounds. Training these models often happens under careful direction.

I. INTRODUCTION

One step at a time, old systems struggled with language. Every task ran on a different blueprint. Getting replies from written words worked fine - yet asking new things required starting over each time. Out of nowhere, connections started shifting shape. These massive models juggle tasks like never before. With transformers came a whole new rhythm - parts clicking in place unlike earlier attempts. A single setup now slides across functions without breaking stride. Narrow components? Replaced by wide-reaching blueprints that adapt on the move [1].

Out of rough blocks of words, queries form - then replies follow close behind. Running steady inside one consistent design, it shifts written stuff into fresh written stuff, no person required once started. What happens next leans on what just passed through. Rather than shaping training bits by hand or pulling old records apart, the system grows its own exact content. Heavy work gets done automatically, yet results stay clear and tight.

Looking back earlier attempts were clunky. The code would be stacked a mile high trying to catch any grammatical rule possible but rules can change and words tend to do whatever people who are saying them want them to do. At first when touching natural language every possible path would be carefully coded which quickly devolved into a massive tangled web of conditions that would fall to any slang. Statistical modeling came to the rescue however it was not until the introduction

of attention mechanisms that machines truly understood how to "read between the lines". Instead of having a coded path which tries to capture every interaction in the given phrase, the machine learns the weight of words in relation to one another and places them on the phrase in which they have interaction. It would learn the relative importance and place those weights upon the phrase with which it has an interaction; it focuses the understanding on larger text spans that just surrounding phrases.

A. Challenges

Out of raw text, pulling questions? Harder than it looks. Machines struggle to grasp meaning behind words. Older systems followed rigid rules - finding key terms was enough, until situations grew complex. Once context shifts, those methods break down fast. Getting sentences to flow naturally trips them up every time. Crafting queries like people do? That kind of rhythm rarely holds when code tries to mimic thought. Most basic setups fail once responses need more than canned lines. Moving past keyword hunts means getting what words actually mean - a hurdle for plenty of models. Long messages or messy reasoning tend to break their flow. That moment when patterns stop being enough? Comprehension has to start there.

B. Contributions

This work tackles the problems by trying something easier, needing less control when making question-answer sets. Starting points appear like this: instead of teaching systems to locate answers, they practice predicting questions - flipping how reading works. After that, a setup operates on its own, grabbing clear subject clues from raw material using NLTK methods before anything shows up. Following that step, a two-part prompting method kicks in, nudging sharp questions forward while stretching short notes into full breakdowns - zero human help needed.

II. BACKGROUND AND DATASET

A. The T5 Architecture

Deep inside this project lives something called T5 [1]. Built by Google, it reshapes each language task into a single format - feed it text, get back text. Rather than using distinct designs for various tasks, all of them move through one path: words show up as a cue, the model works on it, gives back a reply built entirely from more words [1].

Built for tuning directions - this method adjusts how models follow guidance

Picking google/flan-t5-base made sense here [2]. That label "Flan"? It points to how these models train - using guided prompts instead of plain text. Task after task, thousands fed in, shaped into precise challenges [2]. This approach lets even smaller versions follow complex instructions without stumbling. Something shifted in the way lessons were delivered to the model - beyond just material shown. Little by little, logical steps appeared even without huge capacity. This adjustment allowed accuracy in leaner versions. The method of teaching weighed heavier than endless data. Complex chains started making sense when patterns were introduced sooner.

When looking into the details of the attention mechanism you realize it is what is doing the bulk of the work. It is similar to a spotlight being placed on that which is most important and throwing everything else in the text into darkness. Without these long paragraphs can blur into meaningless, wordy mush. T5 uses these heavily in order to process text element relations and determine what words are used to answer the target question. The encoder turns the input text into a dense, large numerical matrix where the relationship between the target answer and the context in the input text is stored as values. This matrix is passed to the decoder which unpacks the matrix and writes out a word to answer the question as the attention mechanisms shine a spotlight upon key elements of the context text.

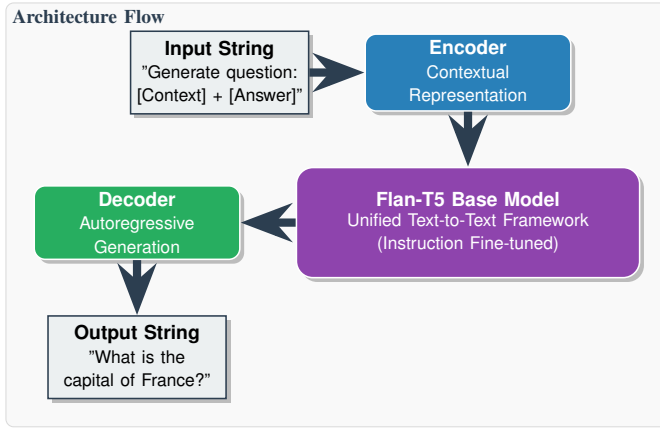


Fig. 1. The Text-to-Text framework mapping diverse inputs to distinct outputs via the Flan-T5 architecture.

B. The Stanford Question Answering Dataset (SQuAD)

The data begins with the SQuAD set [3]. The notable aspects of this dataset are that actual human readers generated the questions after quickly scanning Wikipedia articles. Every question is directly tied to some text within the article and has no interpretations made. Officially called the Stanford Question Answering Dataset, this vast set was designed to "push the performance boundaries of machines in machine reading comprehension". These are not multiple-choice questions, or yes/no style prompts, instead they force machines

to process natural language text directly. The data workers labeled a particular span of text within the article as the correct answer for the question that the human writer penned. Since the answers appear verbatim within the passage it is simple for the machine to locate the relevant text and thereby discover a correct snippet [3].

How is this relevant for our context? SQuAD offers the definitive ground truth for reversing the human thought process. Conventional models process the context and the question to arrive at the answer. The exact opposite occurs here. By providing the system with both the context and the exact span which represents the answer, the system is trained to replicate the human thought process that generated the question. Because SQuAD requires an exact substring as the answer, it is an ideal method for training machines to locate facts within a context [3].

Of the vast amounts of available data, twenty thousand samples are chosen for our specific training set, with an additional five thousand for validation and checking progress. This gives the model a variety of text without making the dataset too large for it to remember without running out of memory while being tuned, and without making the training data too limited. Because the dataset size is controlled, the model is not just memorizing the training data. Also, paragraph lengths in the SQuAD dataset average lengths that are conducive for our Flan-T5 model to process with its 512 token limit.

SQuAD is a little different and it comes with its own set of problems. The context for SQuAD passages are generally not simple, small sentences; it requires the interpretation of multiple dates, people, and places, as well as how they relate to one another. The context of SQuAD is very unforgiving as almost any answer that isn't perfectly correct is considered wrong. For example, if the answer in the context is "May 5th" and the question is "When is it May?", the system would have answered "May" which is not correct, it must answer "May 5th". The system by enforcing this stringent, extraction based, fixed definition of an answer's correctness helps to limit its tendency to "hallucinate" or hypothesize an answer and forces it to retrieve the correct information from the context.

III. METHODOLOGY AND SYSTEM ARCHITECTURE

Out of two key stages, learning kicks things off. Then, once trained, it generates fresh results instead.

A. Data Preprocessing: Inverting the Paradigm

Usually, QA systems take context and a question to produce answers. Turned around, our approach teaches models to generate questions. Starting from a paragraph plus a selected piece, it forms queries like those in SQuAD. Instead of answering, the model shapes the question linked to that fact. With each example, learning shifts through this change in direction.

Out of the twenty thousand examples meant for training - plus an extra five thousand set aside to check progress - comes

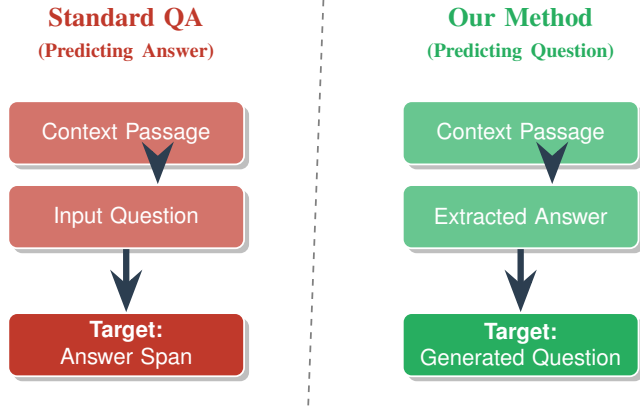


Fig. 2. Comparison of the traditional QA training paradigm versus our inverted question-generation approach.

a custom cleanup stage built with help from Hugging Face’s data tools [4]

Python

Start with flipping the usual approach, then watch Flan-T5 learn how rich prompts lead to single questions. Not by copying examples, yet moving piece by piece - reshaping meaning only from form. With every round, the bond tightens even when no directions are given. Over time, clarity emerges just from seeing enough cases, never from fixed steps.

B. Tokenization and Hyperparameters

At the start, the T5Tokenizer initializes with a MAX_SOURCE_LEN of 512 tokens - plenty for context. Because brevity limits cause issues, setting MAX_TARGET_LEN to 96 opens space for longer answers than SQuAD typically permits.

There is a lot that can go wrong with a model and much of it is dependent on hardware. We cannot ask a machine to analyze millions of phrases at once, so it has to be broken up into batches of limited size so the GPU doesn’t melt. The learning rate also needs to be precisely controlled: too large a learning rate and the loss curve will skyrocket, too small and the training process will take forever. The use of half-precision allows the machine to compute numbers faster while not reducing the accuracy significantly. Finding a good learning rate and training parameters are a delicate balance to supplying a machine with enough data to train without overwhelming it with an excessive amount.

Over at PyTorch, that is where training runs. Setup duties? The Trainer API takes those on [4]. Key choices include:

Learning Rate: 3e-5

Batch Size Four Per Device

Running once over the information. Designed to operate in spots where energy or storage runs thin

Precision at half level activates when the system uses graphics chips to move quicker.

TABLE I
SUMMARY OF FINE-TUNING HYPERPARAMETERS

darkgray		
Base Architecture	google/flan-t5-base params)	(250M
lightgray Training Epochs	1	
Learning Rate	3×10^{-5}	
lightgray Batch Size (Train/Eval)	4 per device	
Max Source Length	512 tokens	
lightgray Max Target Length	96 tokens	
Precision Mapping	FP16 (Mixed Precision)	
lightgray Optimizer	AdamW	

C. Advanced Implementation Details

From time to time, clean patterns pop up in our code layout - patterns that line up well with current ways of building NLP systems Speed jumps up since the code uses the graphics card’s strength through the accelerate library. Instead of waiting, tensors slide onto any available device - CUDA first, if there’s one, or else the regular processor steps in. That swap shaves seconds off both learning and guessing phases, working without noise in the background. One step handles the data while tossing out unneeded columns right away. Memory stays light because of how the prep moves along. Midway through: tensors demand consistent dimensions, meaning any sequence beyond 512 tokens gets trimmed down. If a piece falls short? It’s padded out to hit the same size. PyTorch runs smoothly only when shapes stay identical across samples. While learning unfolds, labels linked to padding spots become -100 instead. This specific value acts like a quiet signal - loss calculations simply skip those positions. Skipping happens on its own during reverse flow. Functions fine even when no added tags or cover-ups step in.

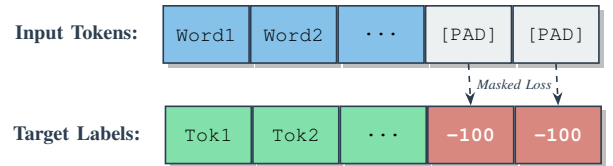


Fig. 3. Visualization of tensor padding and label masking (-100) to optimize cross-entropy loss calculations during PyTorch processing.

Inside the machine numbers are constantly changing. With each passing batch, the loss is reducing. Learning rate parameters are remaining fairly consistent. Gradients are settling out and there are no drastically large spikes or dips. Due to the initial strength of the T5 base model and the addition of instruction tuning, it was expected that the inversion training step would be significantly fast, and so far that is what it seems to be doing. With one pass through the inverted data

the loss is already plummeting and the model is acquiring its new question generation task.

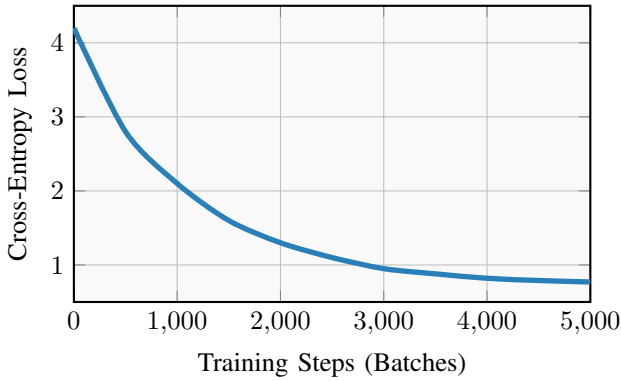


Fig. 4. Training loss curve demonstrating steady convergence over 5,000 steps, showing the model rapidly adapting to the inverted generation task.

IV. THE INFERENCE PIPELINE

Now that tuning wraps up and saves to disk as `qg_model_long`, attention swings to crafting replies using brand new passages - ones never viewed earlier. The actual task kicks in here: pulling questions from untouched text, shaped by nothing that came before.

A. Step 1: Syntactic Candidate Extraction

Before any question can be formed, there must first be a subject. Pulling these subjects out independently led us to create the `extract_candidate_answers` module, relying on the `nltk` library [5].

Splitting the initial text into individual sentences is done using NLTK. After that, each part is managed separately, guided by where speech naturally pauses. The approach mirrors real conversation patterns. Full ideas, not fragments, act as building blocks. Only when splitting ends does analysis begin. Pieces of software such as NLTK shape how breaks happen in text. When the layout follows how people actually talk, things become easier to grasp. Punctuation vanishes first, making space for clarity. What remains are pieces ready to be split apart cleanly. Only digits and alphabets survive this cleanup. Before breaking anything down, the noise is already gone. Nothing lingers where symbols used to sit. Out of the gate, letter-based groupings pull double duty when paired terms stick. Where digits fall short, such pieces earn a spot. Nouns show up often, acting like anchors down the line. One thought nudges the next into place. Strings missing real letters? Left behind. Only clusters formed from proper alphabetic characters move forward.

B. Step 2: Logical Question Generation

From prior stages, any suggested subject moves forward without delay. Following that step, a tailored query enters the modified setup directly. Not every question works elsewhere - it stays tied to its match. Once inside, the enhanced version picks up and proceeds

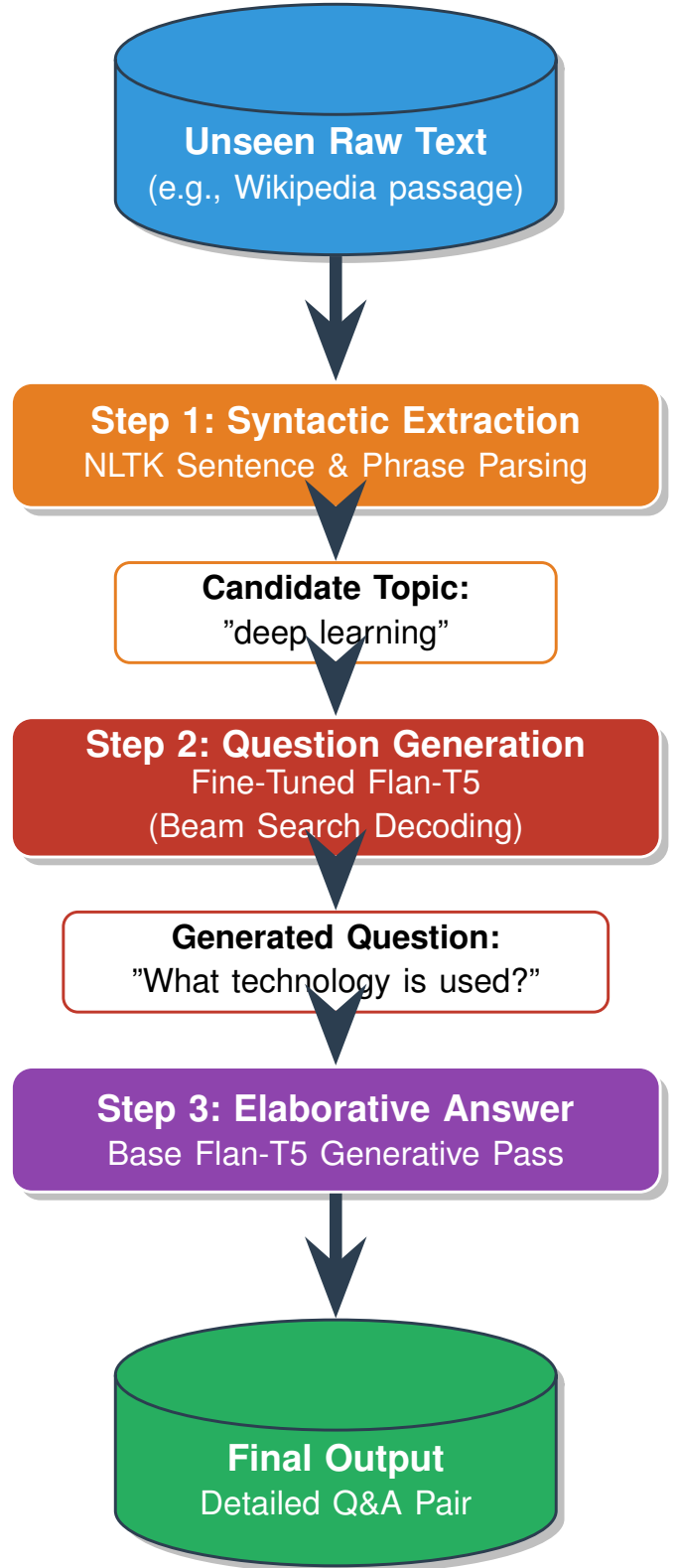


Fig. 5. The three-stage inference pipeline transforming raw passages into structured, elaborative Q&A blocks.

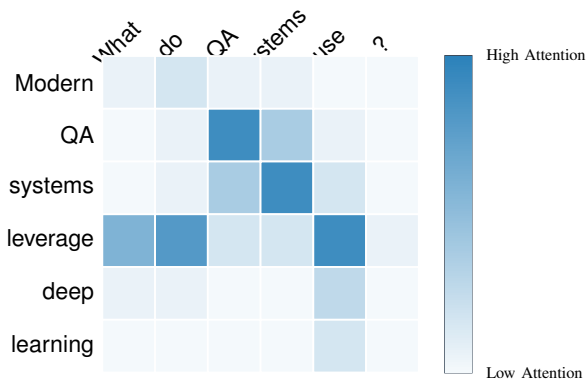


Fig. 6. Cross-attention heatmap visually demonstrating how the decoder aligns generated question tokens (x-axis) with specific input context tokens (y-axis).

Python

Halfway into inference, Beam Search Decoding takes over - running five parallel paths. Not limiting itself to the single most probable word at each step, unlike greedy methods, it weighs multiple solid candidates along the way. The result? Output that feels more coherent, fits context tighter, flows with fewer grammatical hiccups [1]. Greedy decoding rushes ahead and picks the best answer that it has in that moment and moves on, assuming the best choice for that word is the best overall choice. Beam search however takes a more methodical, yet, intelligent approach to generating the answer. Imagine the system can try out five answers at once; beam search is an algorithm that makes the system try out and score five potential answers before selecting one and continues along with the best of those five. While that does use more time it leads to more human-like questions.

C. Step 3: Elaborative Answer Generation

Short bits from the original SQuAD data make up most replies - tiny pieces, sometimes only two or three words. Because these slices miss the depth required for real understanding, an extra stage follows each pull. That follow-up shapes richer answers, crafting broader descriptions rather than copying lines outright.

A new question shows up, tied back to where it began. Sent together, they prompt the main system to stretch its replies. The initial context sticks around, shaping how thoughts develop. Rather than settling for surface level, it goes deeper - expanding what exists. Every move links simply, never piling on too fast. Over time, it circles round without noise, slipping past thoughts again. Step by step, growth shows up - never in one rush

Python

Stretching the limit to 180 lets thoughts run longer. With five beams guiding the search, choices get weighed first. Rather than repeat phrases exactly, replies grow piece by piece. When those controls team up, extended outputs feel more complete. What shows up? Something layered - less copy, more shape.

V. RESULTS & ANALYSIS

From thin air, things get tough once clean setups meet messy, untouched text. A fresh piece about natural language processing arrives - never seen before. Right away, bits such as "Question Answer (QA) generation" begin appearing. Then what follows? Distinct chunks like "Modern QA systems often," pulled directly using the NLTK tool.

From then onward, the updated Flan-T5 model began forming clear, logical questions tied tightly to particular phrases - no human help needed. Starting with the first expression, it generated a precise query such as: "What role in natural language processing involves creating helpful questions independently?"

Later on, because of that initial question, the system began expanding into a more complete reply. Instead of grabbing just a single phrase, it wove together nearby information to form a coherent thought. Surprisingly, the model reads material, identifies what matters most, asks itself a relevant question, and then gives a precise answer - showing just how smoothly reverse teaching fits within its step-by-step method.

Naturally, there are also mistakes that machines make. If the input context contains too many commas, the machine may lose its sense of what is important and it will instead choose something trivial from the text to answer the question. The machine may answer something that is entirely too long due to it trying to pull every single piece of factual data that it can from the text in an effort to answer the question accurately. These errors exhibit where the statistical nature of machine learning causes machine learning to fail in comparison to true human comprehension.

VI. DISCUSSION AND FUTURE WORK

Right now, one tuned transformer handles concept extraction plus question-answer creation just fine. Still, room exists to make things better down the line.

Fitting text into clearly defined chunks is a complex problem. We're essentially told the model not to look past a certain point and so when the limit of 512 tokens comes round it throws the last few words of the sentence away blankly. So if the answer is at the bottom of a long page the model will miss it altogether. For now a workaround to this is a manually chunking text into smaller portions to input into the machine. This could be overcome with a sliding window. Essentially the model scans the text like a scanner, the window of text it has context over moving down the page so that it isn't lost for the thread of the text.

There's also an implicit cost involved in building such systems. Teaching a machine English alone takes months for several large rooms of servers and the sheer amount of energy is enormous. Fortunately there's transfer learning. We never have to tell the machine what a noun is. As the base model Flan-T5 is a machine that is already trained in large volumes of data it can already speak from day one, all it needs is to learn how to question the machine. This would take months normally. Transferring learning saved this weeks and turned what was a power hungry machine server into a computer

TABLE II
QUALITATIVE PIPELINE OUTPUT: SPANS, QUESTIONS, AND ELABORATIVE ANSWERS

darkgray			
“Modern QA systems leverage deep learning to understand context...”	deep learning models	What do modern QA systems rely on to comprehend context?	Modern QA systems rely heavily on deep learning models and transformer architectures to process vast amounts of text, allowing them to understand nuance and context effectively.
lightgray “Chatbots are widely used in educational tools to improve retrieval.”	educational tools	Where are chatbots commonly deployed to enhance information retrieval?	Chatbots are primarily deployed within educational tools and digital search environments. This implementation helps streamline the retrieval of relevant information for users.
“By learning language patterns, systems can produce human-like pairs.”	learning language patterns	How do systems manage to generate human-like question-answer pairs?	By actively learning and analyzing underlying language patterns, these systems adapt to human syntax, allowing them to generate responses that feel natural and accurate.

that could train on a graphics card. This means that anyone is able to train a smart machine they just need a machine others have already created.

Ultimately, the machine is missing human spark. Although it answers with correct grammar and facts it seems like there’s something missing in the questions it forms, there is no real inquisitive nature in the questions compared to a human reading text. A human reads and wants the answer because it is not yet known by the person. A machine asks a question because the weights align with that specific part of the text. Going from statistically derived output to genuine curiosity is a goal that has yet to be reached.

- 1) Start things off by setting up the space with pdfplumber installed. Following that, bringing in tools to read PDFs makes sense - this way, full textbooks or academic papers can flow into the system. From there, study aids could form without needing manual input.

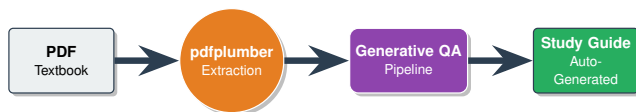


Fig. 7. Proposed future architecture for automated document ingestion (via pdfplumber) directly into the generative QA pipeline.

- 2) Right now, people judge how good the output looks by hand. Moving forward, systems could use tools like BLEU or ROUGE instead - these measure how much the machine-made questions match real human-written ones. Instead of guessing, numbers would show what fits. While these numbers help, other factors that cannot be easily quantifiable exist as well. ROUGE scores attempt to measure how many words are shared between the answer that the system generated and a reference answer; this can provide an insight into how well the text has been integrated with the system’s knowledge, however it does not necessarily indicate how well the written output sounds to the reader.

- 3) Instead of today’s basic n-gram method using NLTK, swapping in a focused NER system might sharpen results by pulling only precise people, places, or happenings. Though the present setup runs on simple patterns, upgrading could mean tighter matches when spotting key topics. A shift like that would center responses around concrete things, not vague phrases. Right now it casts a wide net - switching tools may help narrow what counts as an answer.

VII. CONCLUSION

The project built a full pipeline that turns raw text into questions and answers automatically. Using the google/flan-t5-base model along with the SQuAD data, it flipped how reading tests are usually made by training the machine to create questions instead. With careful phrase selection and two-step prompts, the setup pulls out key parts of sentences then forms clear queries. Instruction-focused transformer models drove this shift, showing they can grow easily across learning materials. Scaling up becomes simpler when machines format knowledge without constant human input. Instead of handcrafting quizzes, systems like this generate them on demand using patterns learned during training.

REFERENCES

- [1] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [2] H. W. Chung *et al.*, “Scaling instruction-finetuned language models,” *arXiv preprint arXiv:2210.11416*, 2022.
- [3] P. Rajpurkar *et al.*, “SQuAD: 100,000+ questions for machine comprehension of text,” *EMNLP*, 2016.
- [4] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” *EMNLP*, 2020.
- [5] S. Bird *et al.*, *Natural Language Processing with Python*, O’Reilly Media, 2009.